NavRoute Package — User Manual

1. Introduction

NavRoute is a navigation extension package for Unity that builds on top of the AI Navigation (NavMesh) system. It provides graph-based route planning, weighted Point of Interest (POI) selection, and advanced pathfinding algorithms for NPC movement.

⚠ **Requirement:** *This package requires the AI Navigation package (Unity's NavMesh system) to be installed in your project.*
You can install it via Window → Package Manager → AI Navigation.

2. Quick Start

Follow these steps to set up a basic NPC navigation system using *NavRoute*:

Step 1: Install AI Navigation

- Open **Window → Package Manager**
- Search for **AI Navigation** and install it
- Bake a NavMesh in your scene via **Window → AI → Navigation → Bake**

Step 2: Import the NavRoute package

- Place the *NavRoute* folder into your *Assets/* directory
- Ensure all scripts compile without errors

Step 3: Create the route system

- Add */Prefabs/RouteService* prefab to your scene

Step 4: Add Poi (Points of Interest)

- Add */Prefabs/Poi* prefab to *Pois* inside the *RouteService* object
- Configure each:
    - *Priority* (0–10)
    - *OnPoiAnimation*
    - *TimeModificator*

Step 5: Add Waypoints

- Add */Prefabs/Waypoint* prefab to *Waypoints* inside the *RouteService* object, if it's needed

Step 6: Create routes

- Add *Active Routes* elements to all the routes you need in the *RouteService* script inside the *RouteService* object.
- Assign:
    - *From* → a *Poi*
    - *To* → another *Poi*
    - *Path* → optional intermediate *Waypoints*

Step 7: Set up the NPC

1. Add a *NavMeshAgent*, *NavMeshObstacle,* and *Animator components* to your NPC
2. Add the *RouteWalker component* to the NPC
3. Assign references:
    - *RouteService*
    - *NavMeshAgent*
    - *NavMeshObstacle*
    - *Animator*
4. Call *StartWalk()* to begin navigation

3. Demo Scene

A **demo scene** is included with the package to illustrate a complete setup:

What the demo shows:

- Multiple *Pois* with different priorities
- Several *Routes* forming a graph
- NPCs walking between points with random stops
- Animation triggers at each *Poi*

To run the demo:

- Open *NavRoute/Demo/Scene/DemoScene.unity*
- Press **Play**
- The NPC will start walking automatically

4. Package Architecture

4.1. Folder Structure

Assets/NavRoute/:

Folder	Description
Demo	Asset demo
• Materials	Materials used in the demo scene
• Prefabs	Prefabs used in the demo scene
• Scene	Demo scene
• DemoScene	Auxiliary files of the demo scene
• Textures	Textures used in the demo scene
Documentation	Asset documentation
Prefabs	Prefabs used in the asset
Runtime	Scripts used in the asset
• Algorithms	Pathfinding algorithms (BFS, A*, Dijkstra, etc.)
• Core	Core entities (Waypoint, Poi, Route, RouteGraph)
• Movement	Agent movement controller (RouteWalker)
• Navigation	Route service and visualization
• Utils	Helper classes (PriorityQueue, ChildCountChangeNotifier)

4.2. Core Components

Component	Description
Waypoint	Basic point on the map (position only).
Poi	Inherits from Waypoint. Includes priority, animation trigger, and time modifier.

Route	An edge connecting two Pois, with optional intermediate Waypoints.
RouteGraph	The graph built from the list of active routes.
RouteService	Main service: builds the graph, selects random routes, finds the nearest Poi, and chooses the algorithm.
RouteWalker	Attached to NPCs. Manages movement, stops, look-at behavior, and animations.
RouteVisualizator	Editor-only tool for visualizing points and routes (Gizmos, Handles).

5. Pathfinding Algorithms

RouteService supports **9 algorithms**, selected via enum *AlgorithmType*.

Algorithm	Type	Best For
DFS	Non-optimal	Fast exploration, debugging
BFS	Optimal	Small graphs, shortest path by edge count
IDDFS	Optimal	DFS with depth limiting
Dijkstra	Optimal	Weighted graphs (by distance)
A*	Optimal	Weighted graphs + heuristic (Euclidean)
BiDDFS / BiBFS / BiDijkstra / BiA*	Optimal	Bidirectional versions — faster on large graphs

Recommendation: For production, use *BiAStar* or *BiDijkstra* for large graphs, and *BFS* for small graphs.

6. RouteWalker API

Methods

```
void StartWalk();                       // Start random route traversal
void StopWalk();                        // Stop all movement and coroutines
void GoTo(Vector3 target, Action onComplete);      // Move to a specific
position
void LookAt(Vector3 target, Action onComplete);    // Rotate toward a position
```

Parameters

Field	Description
_minDelayTimeInPoint	Minimum stop duration at a Poi
_maxDelayTimeInPoint	Maximum stop duration at a Poi
_routeService	Reference to the RouteService
_agent	Reference to NavMeshAgent
_obstacle	Reference to NavMeshObstacle (enabled during stops)
_animator	Reference to Animator

7. Visualization & Debugging

7.1. RouteVisualizator

Attach to RouteService to enable:

- Is Poi Visible — show/hide Pois
- Is Waypoints Visible — show/hide intermediate Waypoints
- Is Ways Visible — show/hide route lines

7.2. Debug Mode

Enable Is Debug on RouteService to log:

- Algorithm execution time (ticks)
- Number of edges in the path
- List of visited points
- Total path length

7.3. Test Point Mode

With Is Debug enabled:

- Set a Test Point (a specific Poi)
- All GetRandomRouteFrom() calls will target that Poi — useful for testing specific routes

8. Extension Guidelines

- Adding new animations → Append to enum PoiAnimations at the end

- Adding custom algorithms → Inherit from PathfindingAlgorithm, implement FindShortestWay(), and add to enum AlgorithmType
- Extending Poi → Add custom events like OnPoiReached for quest/dialogue integration
- Performance → Use BiAStar for large graphs; disable Is Debug in builds

9. Troubleshooting

Issue	Cause	Solution
Agent doesn't move	No *NavMeshAgent* / no NavMesh	Install AI Navigation, bake NavMesh
Agent flies up	Not grounded on NavMesh	Place agent on NavMesh surface
Agent stops too early	*Stopping Distance* too low	Increase *Stopping Distance* on agent
Path not found	Missing *Route* between POIs	Create *Route* objects between points
No rotation	*angularSpeed* = 0	Set *angularSpeed* > 0 on *NavMeshAgent*
Exception at start	No *Pois* in the scene	Add at least one *Poi* child in *RouteService*
Build error	*NavRoute* missing AI Navigation	Install AI Navigation package

10. Summary

NavRoute is a **Unity AI Navigation extension** that adds:

- Graph-based route planning
- Weighted POI selection
- 8 pathfinding algorithms (including bidirectional variants)
- Editor-friendly visualization
- Simple NPC control via RouteWalker
- Demo scene with ready-to-use prefabs

It is designed for games that require **natural NPC behavior** — wandering, patrolling, and interacting with the world — and is easy to extend and customize.

11. Support the Project

If you find this package useful and would like to support its ongoing development, you are welcome to do so on a voluntary basis.

For more details, please refer to the `Support.md` file located in the root folder of the package.

Your support is greatly appreciated and helps keep the project alive and improved over time.